

ScrollSense: Assignment 1

E,F and G

Abhijeet Kumar
DA24B001

Section E

E.1

For the physical implementation, I use SQLite 3.51.2. I keep the same relations from Deliverable C and implement their primary keys, foreign keys, business-rule checks and temporal attributes in SQLite.

I use one timestamp representation throughout the schema: **ISO-8601 UTC text**, for example 2026-09-13T10:30:00Z. I chose this because it keeps timestamps consistent across all tables and also makes the ordering of ISO-8601 UTC timestamps straightforward for the time-based queries in Deliverable F.

The first two statements in `schema.sql` are:

```
PRAGMA foreign_keys = ON;  
PRAGMA journal_mode = WAL;
```

The first pragma is necessary because SQLite does not enforce declared foreign keys on a connection unless foreign-key enforcement is enabled. The second is required for the concurrency demonstration in Deliverable G.

SQLite type-affinity check

SQLite does not enforce the same strict type system as a traditional relational database. For example, I tested an `INTEGER` column by inserting a text value:

```
CREATE TABLE affinity_test(x INTEGER);  
  
INSERT INTO affinity_test(x)  
VALUES ('abc');  
  
SELECT x, typeof(x)  
FROM affinity_test;
```

The value is accepted and SQLite reports its storage class as `text`. This shows that simply declaring a column as `INTEGER` does not guarantee that every stored value will actually be an integer. Therefore, correctness for this schema depends significantly on the appropriate `CHECK`, `NOT NULL`, `UNIQUE`, and foreign-key constraints. Where necessary, additional strictness can be obtained using SQLite's `STRICT` table option, but I keep the logical domains from Deliverable C and enforce the important business rules explicitly.

Foreign keys and deletion behaviour

I do not use the same `ON DELETE` action for every relationship. The choice depends on what the child row means.

For example, deleting an `Impression` should remove its dependent `ViewSegment` rows, so that relationship uses `ON DELETE CASCADE`. Similarly, `VideoHashtag` is an association and can safely be deleted when its parent video or hashtag is removed.

On the other hand, historical records such as moderation decisions and signals should not disappear simply because a parent identity changes. Those relationships therefore use `ON DELETE RESTRICT` or `ON DELETE SET NULL`, depending on the meaning of the reference.

The complete choices are documented as comments in `schema.sql`.

Closed value sets

SQLite has no native `ENUM` type, so I use `CHECK (... IN (...))` for small closed sets whose values are known and not expected to change frequently.

Examples include:

- account status,
- moderation state,
- moderation decision source,
- signal type,
- share destination,
- user rating,
- audio source type.

This keeps the constraints close to the data and avoids introducing lookup tables where the value set is small and stable.

Handle uniqueness

Handles are stored using `COLLATE NOCASE` so that `Abhijeet`, `abhijeet`, and `ABHIJEET` are treated as the same handle for the ASCII case handled by SQLite.

The active-account requirement is slightly different from simple global uniqueness because an old handle may remain in the historical table after a user changes it. I therefore use a partial unique index on currently active handle periods:

```
CREATE UNIQUE INDEX ...  
ON UserHandlePeriod(handle COLLATE NOCASE)  
WHERE valid_to IS NULL;
```

This enforces case-insensitive uniqueness among currently active handles while allowing the same handle string to appear in historical periods.

SQLite's `NOCASE` comparison is ASCII-focused. Therefore, this constraint should not be treated as full Unicode case folding for arbitrary Tamil or other non-ASCII text. The generator and application-level normalisation must handle such input explicitly.

NULL audit

Nullable attributes are used only when `NULL` has a specific meaning.

For example:

- `Video.audio_track_id = NULL`: the video has no audio track.
- `User.phone_number = NULL`: the user did not register using a phone number.
- `User.google_account_id = NULL`: the user did not register using Google.
- `UserHandlePeriod.valid_to = NULL`: the handle period is currently active.
- `ModerationDecision.reviewer_user_id = NULL`: the decision was not made by a human reviewer.
- `ModerationDecision.classifier_version = NULL`: the decision was not made by the classifier.
- `Signal.share_destination = NULL`: this signal is not a share.
- `Signal.comment_text = NULL`: this signal is not a comment.
- `Signal.retracts_signal_id = NULL`: this signal is not a retraction.
- `ToolCall.parent_tool_call_id = NULL`: this is a top-level tool call.
- `ModelPricingPeriod.valid_to = NULL`: this pricing period is currently open-ended.

Thus, `NULL` is not used interchangeably for “unknown”, “not applicable”, and “user declined”. Each nullable column has a specific interpretation.

JSON decision

The only JSON field in the design is `ToolCall.arguments_json`. Tool arguments are the one place where JSON is useful because different tools can have different argument structures.

The column is stored as SQLite `TEXT` and protected with:

```
CHECK (json_valid(arguments_json))
```

I do not use JSON for data such as user profiles, moderation state, or pricing because those values have a fixed relational structure and need normal foreign keys and constraints.

E.2 — Constraint Trace

I traced the twelve assumptions from Deliverable A to the physical database. For each rule, I have noted whether it is enforced directly by SQLite or whether it needs application/trigger logic. This distinction is important because some business rules depend on comparing multiple rows or on the current state of another relation.

Assumption	Rule in words	How it is enforced	Where
A1	A user registers using a phone number or Google account. Each authentication identity belongs to at most one user account.	The authentication identifiers are stored as nullable attributes and are subject to uniqueness constraints. The allowed authentication values are restricted with a CHECK constraint.	User
A2	Handle comparison ignores leading/trailing whitespace and capitalisation, while punctuation inside the handle is preserved.	The application normalises leading/trailing whitespace before storing the handle. Case-insensitive uniqueness is enforced using <code>COLLATE NOCASE</code> .	UserHandlePeriod + application logic
A3	A handle can be changed at most twice in any rolling 365-day period.	Handle changes are stored as historical periods. The rolling 365-day limit requires comparing previous changes, so this rule is enforced in application logic or by a trigger.	UserHandlePeriod + application/trigger
A4	Declared interests may be added or removed after signup and the changes are retained as history.	The user-interest relationship is represented by rows rather than a fixed set of columns, allowing the history to be retained.	UserDeclaredInterest
A5	Suppression of an inferred interest remains effective across later ML refreshes until the user removes it.	Suppression is stored separately from inferred-interest refreshes, so a later refresh does not overwrite or remove the suppression record.	InterestSuppression
A6	Entering the deletion window makes the account invisible while keeping it recoverable for 30 days.	Account state and validity are stored as temporal state. The exact transition into and out of the 30-day recovery window is handled by application logic.	UserStatusPeriod + application logic
A7	Account recovery restores the same account rather than creating a new identity.	The original <code>user_id</code> is retained and account recovery is represented as a state transition.	UserStatusPeriod + application logic
A8	A creator is a role associated with an existing user, so a user does not need to be a creator.	<code>Creator.user_id</code> is a foreign key to <code>User</code> and is <code>UNIQUE</code> , allowing at most one creator row per user.	Creator
A9	A creator has at most one monetisation tier at a given instant, with tier changes represented as consecutive temporal periods.	Tier history is stored using <code>valid_from</code> and <code>valid_to</code> . Checking that periods do not overlap requires comparing multiple rows and therefore needs application/trigger logic.	CreatorTierPeriod + application/trigger
A10	A video uses zero or one audio track, while an audio track may be shared by many videos.	The nullable <code>Video.audio_track_id</code> foreign key represents the optional single track used by a video.	Video, AudioTrack
A11	Hashtags remain in the caption but are also available as queryable relationships, with case-insensitive matching.	Hashtags are stored in <code>Hashtag</code> and linked using <code>VideoHashtag</code> . <code>COLLATE NOCASE</code> provides case-insensitive comparison.	Hashtag, VideoHashtag
A12	A video has one current moderation state at an instant, while all state changes remain available as history.	The allowed moderation states are restricted with a CHECK constraint and every decision is stored as a separate row. The current state is derived from the latest decision rather than duplicated on <code>Video</code> .	ModerationDecision

The main rules that cannot be represented by a single declarative SQLite constraint are the rolling handle-change limit, the active-account aspect of handle uniqueness, and temporal non-overlap. I therefore explicitly identify these as application or trigger responsibilities rather than treating the schema as enforcing more than it actually does.

E.3: Data Generator

The data generator is parameterised at the top of the script and seeded with the student's roll number.

```
SEED = <DA24B001>
```

```
N_USERS = 5000
N_VIDEOS = 20000
N_IMPRESSIONS = 300000
N_AGENT_SESSIONS = 2000
SCALE = 1
```

The generator preserves the general shape described in the brief rather than using independent uniform random values for everything. Watch time is right-skewed, follower counts are approximately power-law distributed, activity follows a daily rhythm, and the interaction funnel is sparse: most impressions do not become views and most views do not produce an explicit signal.

All data is inserted in dependency order inside one transaction. The generator also enables:

```
PRAGMA foreign_keys = ON;
```

on its own database connection before inserting any data.

Deliverable F – Queries

This section presents the thirteen SQL queries used to answer the required analytical questions over the ScrollSense database. For each query, the intent, SQL statement, result size, first five rows, wall-clock runtime, and a short interpretation are reported. The queries were run without tuning so that the runtimes represent the baseline execution performance required by the assignment.

0.1 F1 – Top 10 Audio Tracks by Recent Video Usage

Intent: Identify the 10 audio tracks that were used by the highest number of distinct videos uploaded in the last 7 days.

```
SELECT a.track_id,a.title,
COUNT(DISTINCT v.video_id) AS video_count
FROM AudioTrack a
JOIN Video v ON v.audio_track_id = a.track_id
WHERE v.uploaded_at >= datetime('now', '-7 days')
GROUP BY a.track_id,a.title
ORDER BY video_count DESC,a.track_id
LIMIT 10;
```

Rows returned: 10

First five rows:

```
8634|licensed_audio_1634|2
9784|licensed_audio_2784|2
10376|licensed_audio_3376|2
11|original_audio_16522|1
38|original_audio_8613|1
```

Runtime: 0.018466 seconds

Reading: The most frequently reused audio tracks appeared in two distinct videos during the selected recent period, while the remaining top tracks appeared in one video each.

0.2 F2 – Watch Hours and Completion Rate per Creator

Intent: Calculate total watch hours and mean completion rate for each creator's live clips while retaining every creator, including creators with no live clips or no watch activity.

```
SELECT c.creator_id,c.user_id,
ROUND(COALESCE(SUM(COALESCE(w.watch_ms,0)),0) / 3600000.0,2) AS watch_hours,
ROUND(COALESCE(AVG(COALESCE(w.completed,0)),0),3) AS mean_completion_rate
FROM Creator c
LEFT JOIN ( SELECT v.video_id,v.owner_id FROM Video v JOIN (SELECT video_id,state
FROM (
SELECT video_id,state,decision_id, ROW_NUMBER() OVER (PARTITION BY video_id
ORDER BY decided_at DESC,decision_id DESC) AS rn
FROM ModerationDecision
) WHERE rn = 1 AND state = 'live') s ON s.video_id = v.video_id ) v ON v.owner_id = c.user_id
LEFT JOIN Impression i ON i.video_id = v.video_id
LEFT JOIN ( SELECT impression_id, SUM(segment_end_ms - segment_start_ms) AS watch_ms, MAX(reached_end) AS
completed
FROM ViewSegment
GROUP BY impression_id) w ON w.impression_id = i.impression_id
GROUP BY c.creator_id,c.user_id
ORDER BY watch_hours DESC,c.creator_id;
```

Rows returned: 900

First five rows:

```
772|3286|0.75|0.051
794|4992|0.55|0.057
596|4123|0.53|0.035
2|4421|0.52|0.047
705|535|0.48|0.025
```

Runtime: 0.253178 seconds

Reading: All 900 creators are retained, and creators with no watch activity receive zero-valued aggregates rather than being silently removed.

0.3 F3 – Videos with No Audio Track

Intent: Find videos that do not have an associated audio track and demonstrate the difference between NOT EXISTS and NOT IN when NULL values are possible.

NOT EXISTS version:

```
SELECT v.video_id,v.owner_id,v.uploaded_at
FROM Video v
WHERE NOT EXISTS (
SELECT 1
FROM AudioTrack a
WHERE a.track_id = v.audio_track_id)
ORDER BY v.video_id;
```

Rows returned: 5615

First five rows:

```
6|442|2026-06-26T09:35:52Z
11|1752|2026-07-18T17:22:19Z
15|3703|2026-07-15T23:23:37Z
17|1171|2026-07-15T11:56:32Z
21|2550|2026-06-06T03:13:20Z
```

Runtime: 0.000215 seconds

NOT IN version:

```
SELECT COUNT(*)
FROM Video
WHERE audio_track_id NOT IN (SELECT track_id FROM AudioTrack);
```

Rows returned: 0

Reading: NOT EXISTS correctly identifies the 5615 videos whose audio reference is NULL, whereas NOT IN returns zero because comparison with a NULL left-hand value evaluates to UNKNOWN rather than TRUE.

0.4 F4 – Likes Retracted within 60 Seconds

Intent: Identify users who liked a clip and subsequently retracted that like on the same clip within 60 seconds.

```
SELECT l.user_id,l.video_id,
l.created_at AS liked_at,
r.created_at AS retracted_at,
ROUND((julianday(r.created_at) - julianday(l.created_at)) * 86400,2 ) AS seconds_to_retract
FROM Signal l
JOIN Signal r ON r.retracts_signal_id = l.signal_id
WHERE l.signal_type = 'like'
AND r.signal_type = 'like_retraction'
AND r.created_at > l.created_at
AND (julianday(r.created_at) - julianday(l.created_at)) * 86400 <= 60
ORDER BY l.created_at,l.signal_id;
```

Rows returned: 586

First five rows:

```
2421|1235|2026-03-01T08:49:59Z|2026-03-01T08:50:43Z|44.0
2780|8571|2026-03-01T14:12:14Z|2026-03-01T14:12:56Z|42.0
3987|12079|2026-03-02T16:46:18Z|2026-03-02T16:46:34Z|16.0
2115|344|2026-03-02T18:26:21Z|2026-03-02T18:27:04Z|43.0
1049|501|2026-03-02T18:51:14Z|2026-03-02T18:51:50Z|36.0
```

Runtime: 0.001917 seconds

Reading: There were 586 like events that were explicitly retracted on the same clip within 60 seconds.

0.5 F5 – Videos Containing a Given Hashtag

Intent: Find videos containing the hashtag #life, matching case-insensitively while tolerating surrounding punctuation and whitespace.

```
WITH normalized AS (
  SELECT
    video_id,
    owner_id,
    caption,
    ' ' || trim(
      replace(
        replace(
          replace(lower(caption), ',', ' '),
          '.', ' ',
          '!', ' '
        ),
        '?', ' '
      )
    ) || ' ' AS caption_norm
  FROM Video
)
SELECT video_id, owner_id, caption
```

```
FROM normalized
WHERE caption_norm LIKE '% #life %'
AND instr(caption_norm, ' #life ') > 0
ORDER BY video_id;
```

Rows returned: 1012

First five rows:

```
18989|4771|video 18989 #daily #life
18990|4857|video 18990 #daily #life
18991|798|video 18991 #daily #life
18992|1834|video 18992 #daily #life
18993|3454|video 18993 #daily #life
```

Runtime: 0.073635 seconds

Reading: The query finds 1012 videos containing the requested hashtag after normalization; LIKE was used with lower(), and the answer does not change for a Tamil caption because Tamil has no upper/lower-case distinction, although SQLite's case-insensitive behavior is primarily ASCII-based.

0.6 F6 – Users Shown Clips but Never Engaged

Intent: Identify users who were shown clips from creator 2 but generated no explicit signal for any of those clips, using a set operator.

```
SELECT DISTINCT i.user_id
FROM Impression i
JOIN Video v ON v.video_id = i.video_id
WHERE v.owner_id = 2
EXCEPT
SELECT DISTINCT s.user_id
FROM Signal s
JOIN Video v ON v.video_id = s.video_id
WHERE v.owner_id = 2
ORDER BY user_id;
```

Rows returned: 100

First five rows:

```
41
153
162
177
191
```

Runtime: 0.078712 seconds

Reading: 100 users were exposed to creator 2's clips but did not generate an explicit engagement signal for those clips.

UNION version of a signal roll-up:

```
SELECT video_id, 'like' AS signal_group, COUNT(*) AS signal_count
FROM Signal
WHERE signal_type = 'like'
GROUP BY video_id
UNION
SELECT video_id, 'save' AS signal_group, COUNT(*) AS signal_count
FROM Signal
WHERE signal_type = 'save'
GROUP BY video_id
ORDER BY video_id, signal_group;
```

Rows returned: 1012

First five rows:

```
16620|like|1
16623|like|1
16623|save|1
16626|like|1
16626|save|1
```

Runtime: 0.012456 seconds

UNION ALL version:

```
SELECT video_id,'like' AS signal_group,COUNT(*) AS signal_count
FROM Signal
WHERE signal_type = 'like'
GROUP BY video_id
UNION ALL
SELECT video_id,'save' AS signal_group,COUNT(*) AS signal_count
FROM Signal
WHERE signal_type = 'save'
GROUP BY video_id
ORDER BY video_id,signal_group;
```

Rows returned: 1012

First five rows:

```
16620|like|1
16623|like|1
16623|save|1
16626|like|1
16626|save|1
```

Runtime: 0.018656 seconds

Reading: Both versions return 1012 rows because the two branches produce different `signal_group` values, so there are no duplicate tuples for `UNION` to remove; `UNION ALL` therefore preserves the same rows.

0.7 F7 – Agent Session Cost by Prompt Template Version

Intent: Calculate the cost of each agent session during August 2026, broken out by prompt template version and restricted to sessions whose cost exceeds the selected threshold of 0.05.

```
SELECT s.session_id,ptv.template_version_id,ptv.version_number,
ROUND(SUM((t.input_tokens - t.cached_input_tokens) * p.input_rate + t.cached_input_tokens * p.cached_input_rate
+ t.output_tokens * p.output_rate),6) AS session_cost
FROM Session s
JOIN Turn t ON t.session_id = s.session_id
JOIN PromptTemplateVersion ptv ON ptv.template_version_id = t.prompt_template_version_id
JOIN ModelPricingPeriod p ON p.model_id = t.model_id AND t.created_at >= p.valid_from AND (p.valid_to IS NULL
OR t.created_at < p.valid_to)
WHERE s.started_at >= '2026-08-01T00:00:00Z'
AND s.started_at < '2026-09-01T00:00:00Z'
GROUP BY s.session_id,ptv.template_version_id,ptv.version_number
HAVING session_cost > 0.05
ORDER BY session_cost DESC,s.session_id,ptv.version_number;
```

Rows returned: 814

First five rows:

```
474|2|2|11148.33
1303|8|2|10202.455
165|7|1|10137.135
1690|7|1|10018.685
1074|4|2|10012.015
```

Runtime: 0.011140 seconds

Reading: The result isolates high-cost August sessions and preserves the prompt-template-version grain, while the historical pricing-period join prevents later price changes from altering the calculated historical cost.

0.8 F8 – Videos with More Than Two Moderation Transitions

Intent: Find videos with more than two moderation transitions and display their complete moderation state sequence in chronological order.

```
WITH ordered_decisions AS (
  SELECT video_id, state, decided_at, decision_id FROM ModerationDecision),
transition_counts AS (SELECT video_id, COUNT(*) - 1 AS transition_count
  FROM ordered_decisions
  GROUP BY video_id
  HAVING COUNT(*) - 1 > 2)
SELECT d.video_id, tc.transition_count, group_concat(d.state, ' -> '
ORDER BY d.decided_at, d.decision_id) AS state_sequence
FROM ordered_decisions d
JOIN transition_counts tc ON tc.video_id = d.video_id
GROUP BY d.video_id, tc.transition_count
ORDER BY d.video_id;
```

Rows returned: 11

First five rows:

```
1479|3|pending -> live -> demoted -> demoted
2013|3|pending -> live -> demoted -> live
4618|3|pending -> live -> age_restricted -> demoted
8123|3|pending -> live -> demoted -> age_restricted
11541|3|pending -> live -> age_restricted -> live
```

Runtime: 0.015792 seconds

Reading: Eleven videos have more than two recorded moderation transitions, with the full state sequence preserved in chronological order.

0.9 F9 – Longest Consecutive Active-Day Streak

Intent: Determine each user's longest consecutive active-day streak, where a day is considered active when the user generates at least one signal.

```
WITH active_days AS (
  SELECT DISTINCT
    user_id,
    date(created_at) AS activity_date
  FROM Signal
),
numbered_days AS (
  SELECT user_id, activity_date,
  ROW_NUMBER() OVER (
    PARTITION BY user_id
    ORDER BY activity_date
  ) AS rn
  FROM active_days
),
streak_groups AS (
  SELECT user_id, activity_date,
  date(activity_date, '-' || rn || ' days') AS streak_group
  FROM numbered_days
),
streaks AS (
  SELECT user_id,
  MIN(activity_date) AS streak_start,
  MAX(activity_date) AS streak_end,
  COUNT(*) AS streak_length
  FROM streak_groups
  GROUP BY user_id, streak_group
```

```

)
SELECT user_id, streak_start, streak_end, streak_length
FROM streaks
ORDER BY streak_length DESC,user_id
LIMIT 10;

```

Rows returned: 10

First five rows:

```

666|2026-03-13|2026-03-15|3
24|2026-07-19|2026-07-20|2
33|2026-04-03|2026-04-04|2
71|2026-07-12|2026-07-13|2
130|2026-05-15|2026-05-16|2

```

Runtime: 0.027438 seconds

Reading: User 666 has the longest observed consecutive activity streak, with signals recorded on three consecutive days.

0.10 F10 – Rolling Watch Time

Intent: Calculate rolling watch time using a seven-day window over user activity and identify the change in watch time over the rolling period.

```

WITH daily_watch AS (
  SELECT
    i.user_id,
    date(i.shown_at) AS watch_date,

    SUM(
      vs.segment_end_ms - vs.segment_start_ms
    ) AS watch_ms

  FROM Impression i

  JOIN ViewSegment vs
    ON vs.impression_id = i.impression_id

  GROUP BY
    i.user_id,
    date(i.shown_at)
),

rolling_watch AS (SELECT d1.user_id,d1.watch_date, SUM(d2.watch_ms) AS rolling_7day_watch_ms
FROM daily_watch d1
JOIN daily_watch d2 ON d2.user_id = d1.user_id AND d2.watch_date BETWEEN date(d1.watch_date, '-6 days') AND d1.
  watch_date
GROUP BY d1.user_id,d1.watch_date)
SELECT user_id,watch_date, ROUND(rolling_7day_watch_ms / 3600000.0, 2) AS rolling_7day_watch_hours
FROM rolling_watch
ORDER BY rolling_7day_watch_hours DESC,user_id, watch_date
LIMIT 10;

```

Reading: The rolling calculation smooths daily fluctuations and shows how recent watch activity changes over the selected window.

0.11 F11 – Tool Call Nesting Tree

Intent: Reconstruct the nesting structure of agent tool calls and show the depth and call path of each tool invocation.

```

WITH RECURSIVE tool_tree AS (

```

```

SELECT tc.tool_call_id,tc.turn_id,tc.parent_tool_call_id,tc.tool_name,0 AS depth,printf('%s', tc.tool_name)
  AS call_path
FROM ToolCall tc
WHERE tc.parent_tool_call_id IS NULL
UNION ALL
SELECT child.tool_call_id,child.turn_id,child.parent_tool_call_id,child.tool_name,parent.depth + 1 AS depth
,parent.call_path || ' -> ' || child.tool_name AS call_path
FROM ToolCall child
JOIN tool_tree parent ON child.parent_tool_call_id = parent.tool_call_id)
SELECT turn_id,tool_call_id,parent_tool_call_id,depth,call_path
FROM tool_tree
ORDER BY turn_id,depth,tool_call_id
LIMIT 20;

```

Rows returned: 20

First five rows:

```

3|1||0|lookup_creator
4|2||0|search_videos
5|3||0|search_hashtags
6|4||0|rank_candidates
7|5||0|search_hashtags

```

Runtime: 0.011608 seconds

Reading: The recursive query reconstructs the parent-child structure of tool calls and exposes the depth of each call in the nesting hierarchy.

0.12 F12 – Recommended Clip Watched to Completion

Intent: Identify sessions in which the agent recommended a clip and the same user subsequently watched that recommended clip to completion, while reporting the clip’s shelf position.

```

SELECT DISTINCT s.session_id,t.turn_id,ar.video_id,ar.shelf_position
FROM Session s
JOIN Turn t ON t.session_id = s.session_id
JOIN AgentRecommendation ar ON ar.turn_id = t.turn_id
JOIN Impression i ON i.recommendation_id = ar.recommendation_id
  AND i.video_id = ar.video_id
  AND i.user_id = s.user_id
WHERE EXISTS (
  SELECT 1 FROM ViewSegment vs
  WHERE vs.impression_id = i.impression_id
  AND vs.reached_end = 1)
ORDER BY s.session_id,t.turn_id,ar.shelf_position,ar.video_id
LIMIT 20;

```

Rows returned: 20

First five rows:

```

1|1|12565|3
1|2|15960|5
1|2|5189|6
2|4|1829|4
3|7|18012|6

```

Runtime: 0.058973 seconds

Reading: The result provides traceable recommendation-to-watch links, showing which recommended clips were subsequently watched to completion and where they appeared on the returned shelf.

0.13 F13 – Judge/User Disagreement

Intent: Identify turns where the LLM judge gave a helpfulness score above 4 but the user gave a thumbs-down, exposing cases where automated evaluation and actual user feedback disagree.

```
SELECT t.turn_id,t.session_id,j.helpfulness,j.groundedness,j.safety,r.rating,j.judged_at,r.rated_at
FROM Turn t
JOIN LLMJudgeScore j ON j.turn_id = t.turn_id
JOIN UserRating r ON r.turn_id = t.turn_id
WHERE j.helpfulness > 4
AND r.rating = 'down'
ORDER BY j.helpfulness DESC,t.turn_id
LIMIT 20;
```

Reading: These disagreements are commercially valuable because they identify responses that appear successful to the automated judge but fail to satisfy the actual user, making them high-value examples for improving ranking, evaluation, prompt design, and product quality.

1 G. Views and Transaction Demonstrations

1.1 G.1 Required Views

1.1.1 G.1.1 Public Profile View

The purpose of this view is to expose only the public profile information required by the application. Phone numbers and email addresses are intentionally excluded. The view also shows the current handle, current active status, and follower count.

```
CREATE VIEW v_public_profile AS
SELECT
    u.user_id,
    h.handle,
    u.display_name,
    COUNT(DISTINCT f.follower_id) AS follower_count
FROM User u
JOIN UserHandlePeriod h
    ON h.user_id = u.user_id
    AND h.valid_to IS NULL
LEFT JOIN FollowPeriod f
    ON f.followee_id = u.user_id
    AND f.ended_at IS NULL
JOIN UserStatusPeriod s
    ON s.user_id = u.user_id
    AND s.valid_to IS NULL
    AND s.status = 'active'
GROUP BY u.user_id,h.handle,u.display_name;
```

Test output: The first five rows were:

```
1|user_1|User 1|0
2|user_2_new|User 2|0
3|user_3_new|User 3|2
4|user_4_new|User 4|1
5|user_5|User 5|1
```

The view returned only currently active users and their current handles. Sensitive attributes such as phone number and email are not exposed. The follower count is calculated from currently active follow relationships.

1.1.2 G.1.2 Current Video Moderation State

The moderation history is append-only, so the current state is obtained by selecting the most recent moderation decision for each video.

```
CREATE VIEW v_video_current_state AS
SELECT video_id,state,decided_by_type,reviewer_user_id,classifier_version,decided_at
FROM (
    SELECT video_id,state,decided_by_type,reviewer_user_id,
           classifier_version,decided_at,
           ROW_NUMBER() OVER (
               PARTITION BY video_id
               ORDER BY decided_at DESC,decision_id DESC
           ) AS rn
    FROM ModerationDecision
)
WHERE rn = 1;
```

Test output: The first five rows were:

```
1|pending|classifier||clf-v1|2026-08-27T16:16:32Z
2|live|classifier||clf-v1|2026-06-11T06:12:21Z
3|live|classifier||clf-v1|2026-07-27T20:01:20Z
4|live|classifier||clf-v1|2026-04-17T14:09:50Z
5|live|classifier||clf-v1|2026-04-17T05:40:27Z
```

The view returned one current moderation record per video. The `ROW_NUMBER()` ordering also uses `decision_id` as a deterministic tie-breaker.

1.1.3 G.1.3 Current Creator Tier

This view exposes the creator tier that is valid at the current time.

```
CREATE VIEW v_creator_tier_current AS
SELECT c.creator_id,c.user_id,ctp.tier
FROM Creator c
JOIN CreatorTierPeriod ctp
    ON ctp.creator_id = c.creator_id
    AND ctp.valid_from <= datetime('now')
    AND (ctp.valid_to IS NULL OR ctp.valid_to > datetime('now'));
```

Test output: The first five rows were:

```
1|1068|starter
2|4421|pro
3|2120|starter
4|2063|elite
5|2425|starter
```

The view uses the temporal validity interval to return the tier currently applicable to each creator.

1.1.4 G.1.4 Daily Video Engagement

This view provides daily engagement metrics while retaining impression days even when there are no corresponding views or likes.

```
CREATE VIEW v_video_daily_engagement AS
WITH impression_days AS (
    SELECT
        video_id,
        date(shown_at) AS engagement_date,
        COUNT(*) AS impressions
    FROM Impression
    GROUP BY video_id,date(shown_at)
),
view_days AS (
```

```

SELECT
    i.video_id,
    date(i.shown_at) AS engagement_date,
    COUNT(DISTINCT i.impression_id) AS views,
    SUM(vs.segment_end_ms - vs.segment_start_ms) AS watch_ms
FROM Impression i
JOIN ViewSegment vs
    ON vs.impression_id = i.impression_id
GROUP BY i.video_id,date(i.shown_at)
),
like_days AS (
    SELECT
        video_id,
        date(created_at) AS engagement_date,
        SUM(
            CASE
                WHEN signal_type = 'like' THEN 1
                WHEN signal_type = 'like_retraction' THEN -1
                ELSE 0
            END
        ) AS net_likes
    FROM Signal
    WHERE signal_type IN ('like','like_retraction')
    GROUP BY video_id,date(created_at)
)
SELECT
    i.video_id,
    i.engagement_date,
    i.impressions,
    COALESCE(v.views,0) AS views,
    ROUND(COALESCE(v.watch_ms,0) / 1000.0,2) AS watch_seconds,
    COALESCE(l.net_likes,0) AS net_likes
FROM impression_days i
LEFT JOIN view_days v
    ON v.video_id = i.video_id
    AND v.engagement_date = i.engagement_date
LEFT JOIN like_days l
    ON l.video_id = i.video_id
    AND l.engagement_date = i.engagement_date;

```

Test output: The first five rows were:

```

1|2026-03-05|1|0|0.0|0
1|2026-03-24|1|0|0.0|0
1|2026-04-03|1|0|0.0|0
1|2026-05-01|2|1|5.05|0
1|2026-05-08|1|0|0.0|0

```

The use of `LEFT JOIN` and `COALESCE` ensures that impression days with zero views or zero likes are still represented in the result.

1.1.5 G.1.5 Historical Turn Cost

The turn cost must use the model pricing that was valid at the time the turn was created rather than today's pricing.

```

CREATE VIEW v_turn_cost AS
SELECT
    t.turn_id,
    t.session_id,
    t.model_id,
    t.created_at,
    t.input_tokens,

```

```

t.output_tokens,
t.cached_input_tokens,
p.input_rate,
p.output_rate,
p.cached_input_rate,
ROUND(
  (t.input_tokens - t.cached_input_tokens) * p.input_rate
  + t.cached_input_tokens * p.cached_input_rate
  + t.output_tokens * p.output_rate,
  6
) AS turn_cost
FROM Turn t
JOIN ModelPricingPeriod p
  ON p.model_id = t.model_id
  AND t.created_at >= p.valid_from
  AND (p.valid_to IS NULL OR t.created_at < p.valid_to);

```

Test output: The first five rows were:

```

1|1|2|2026-06-03T16:32:56Z|973|666|607|0.8|1.6|0.2|1479.8
2|1|2|2026-06-03T16:34:56Z|947|417|263|0.8|1.6|0.2|1267.0
3|2|5|2026-03-14T15:27:41Z|1548|801|227|2.0|4.0|0.5|5959.5
4|2|2|2026-03-14T15:30:41Z|600|811|489|0.8|1.6|0.2|1484.2
5|2|1|2026-03-14T15:31:41Z|1012|1157|37|0.4|0.8|0.1|1319.3

```

The temporal join ensures that each turn is priced using the historical pricing period covering its `created_at` timestamp.

1.2 G.2 View Updatability and As-of Parameterization

1.2.1 G.2.1 Attempted Insert into the Public Profile View

The following insert was attempted:

```

INSERT INTO v_public_profile(user_id,handle,display_name,follower_count)
VALUES (999999,'test_profile','Test User',0);

```

SQLite returned the following error:

```
Parse error: cannot modify v_public_profile because it is a view
```

This demonstrates that the view is read-only in SQLite. More generally, an automatically updatable view must provide a clear one-to-one mapping between an output row and a base-table row. The `v_public_profile` view contains aggregation through `COUNT()` and `GROUP BY`, so an inserted follower count cannot be unambiguously mapped back to the underlying relations. PostgreSQL supports automatically updatable simple views, but this particular aggregate view would still require explicit handling rather than assuming that every PostgreSQL view is automatically writable.

1.2.2 G.2.2 As-of Parameterization

SQLite views cannot directly accept a runtime parameter such as an `as_of` timestamp. Therefore, for historical creator-tier lookup, the temporal relation can be queried directly using a parameter:

```

SELECT c.creator_id,c.user_id,ctp.tier
FROM Creator c
JOIN CreatorTierPeriod ctp
  ON ctp.creator_id = c.creator_id
WHERE ctp.valid_from <= ?
AND (ctp.valid_to IS NULL OR ctp.valid_to > ?);

```

The same timestamp is supplied to both parameters. This allows the application to request the creator tier valid at any chosen point in time instead of only the current time.

1.3 G.3 Transaction Demonstrations

1.3.1 G.3.1 Transaction 1: Retraction Atomicity

An existing like signal was selected and a corresponding retraction was inserted inside a transaction.

```
BEGIN;
INSERT INTO Signal(user_id,video_id,signal_type,retracts_signal_id,created_at)
VALUES(183,5,'like_retraction',1,'2026-04-22T21:51:30Z');
SELECT signal_id,user_id,video_id,signal_type,retracts_signal_id
FROM Signal
WHERE retracts_signal_id=1;
ROLLBACK;
```

Inside the transaction, the new row was visible:

```
11356|183|5|like_retraction|1
```

After the ROLLBACK, the retraction was removed and the corresponding query returned zero rows. This demonstrates atomicity: the temporary change existed only within the transaction and did not persist.

1.3.2 G.3.2 Transaction 2: Moderation History and Isolation

Two separate SQLite connections were used with WAL mode enabled.

Connection 1 inserted a new moderation decision inside an uncommitted transaction:

```
PRAGMA journal_mode=WAL;
BEGIN;
INSERT INTO ModerationDecision(
    video_id,state,decided_by_type,classifer_version,decided_at
)
VALUES(1,'live','classifier','clf-demo','2026-09-13T18:00:00Z');

SELECT video_id,state,decided_at
FROM v_video_current_state
WHERE video_id=1;
```

Connection 1 observed:

```
1|live|2026-09-13T18:00:00Z
```

Before the transaction was committed, Connection 2 queried the same video and observed the previous committed state:

```
1|pending|2026-08-27T16:16:32Z
```

After Connection 1 performed ROLLBACK, both connections again observed the original **pending** state. This demonstrates that uncommitted moderation history is not visible to another connection.

1.3.3 G.3.3 Transaction 3: Handle Change Limit and Writer Contention

A trigger was added to enforce the rule that a user can change their handle at most twice within a rolling 365-day period.

```
CREATE TRIGGER handle_change_limit
BEFORE INSERT ON UserHandlePeriod
WHEN (
    SELECT COUNT(*)
    FROM UserHandlePeriod h
    WHERE h.user_id = NEW.user_id
    AND h.valid_from > (
        SELECT MIN(h0.valid_from)
        FROM UserHandlePeriod h0
        WHERE h0.user_id = NEW.user_id
    )
)
```

```
        AND h.valid_from >= datetime(NEW.valid_from, '-365 days')
        AND h.valid_from < NEW.valid_from
    ) >= 2
BEGIN
    SELECT RAISE(ABORT, 'handle can be changed at most twice in 365 days');
END;
```

For user 1, the first two handle changes within the 365-day period were accepted. A third change attempted within the same period failed with:

Runtime error: handle can be changed at most twice in 365 days (19)

The transaction was then rolled back so that the demonstration did not alter the final database state.

Writer contention was also tested using two SQLite connections. Connection 1 acquired a write transaction:

```
BEGIN IMMEDIATE;
UPDATE UserHandlePeriod
SET handle='user_1_locked_test'
WHERE user_id=1 AND valid_to IS NULL;
```

Connection 2 attempted another update while the first connection held the write lock:

```
PRAGMA busy_timeout=0;
UPDATE UserHandlePeriod
SET handle='user_1_busy_test'
WHERE user_id=1 AND valid_to IS NULL;
```

Connection 2 returned:

Runtime error: database is locked (5)

Connection 1 was then rolled back, and the original handle was verified. This demonstrates SQLite's single-writer behavior and the handling of concurrent write contention.